Folha de Respostas -Aula 5

Nome do Aluno: Felipe Uehara Gondo **No. USP: 13680612**

Instruções: Completar com apenas as informações e dados solicitados, e apenas nos campos adequados. A enumeração abaixo segue a da apostila prática.

A) Para os arquivos de imagem subidos, nomeie-os com a enumeração do item correspondente.

B) No Wave, para uma melhor apresentação, ajuste os valores dos sinais/variáveis para Unsigned ou Decimal

1) Rc_adder

c) Insira no quadro abaixo a descrição da arquitetura em VHDL do novo RC_adder (com generate)

```
architecture structural of rc_adder_2 is



    COMPONENT full_adder_1
        port ( a_in, b_in, c_in     :   in STD_LOGIC;
                z_out, c_out        :   out STD_LOGIC);
    END COMPONENT;



signal carry    : STD_LOGIC_VECTOR (WIDTH-1 downto 0);  -- auxiliary signal
carry(x) means carry_out of stage x

begin

-- a seguir, template e versao incompleta

-- FOR generate para todos os bits

    G_ALL : FOR N IN WIDTH-1 DOWNTO 0 GENERATE

--      IF generate para caso LSB- codificar para N=0
        G_LSB: IF (N = 0) generate
            fa_LSB: full_adder_1 port map(  a_in => a_i(N),
                                            b_in => b_i(N),
                                            c_in => '0',
                                            z_out => z_out(N),
                                            c_out => carry(N));
```

```
            end generate G_LSB;
    --      fechar IF



    --      IF generate para caso MSB- codificar para N=WIDTH-1=7
            G_MSB: IF(N = WIDTH-1) generate
                fa_MSB: full_adder_1 port map(  a_in => a_i(N),
                                                b_in => b_i(N),
                                                c_in => carry(N-1),
                                                z_out => z_out(N),
                                                c_out => carry(N));
            end generate G_MSB;
    --      fechar IF


    --      IF generate para demais casos

                 G_CENTRAL  :  IF  (N  <  WIDTH-1  AND  N  >  0)  GENERATE  --  para
n=6,5,4,3,2,1
                fa_central : full_adder_1 port map( a_in => a_i(N),
                                                b_in => b_i(N),
                                                c_in => carry(N-1),
                                                z_out => z_out(N),
                                                c_out => carry(N));
                --full_adder_1 PORT MAP (a_i(N), b_i(N), carry (N-1), z_out(N),
carry (N));
            END GENERATE G_CENTRAL;

    --      fechar IF

        END GENERATE G_ALL;

    -- fechar FOR

    end structural;
```

n) Insira no quadro abaixo o trecho da arquitetura do módulo stimuli_module, em que fiquem identificadas as mudanças por você propostas para a verificação do funcionamento do rc adder.

```
-- test vectors application
--127+53=180
assign_input_words(53, 127);
assign_carry_in('0');

--N = 13680612mod256 = 228
--N + 127 + 0 = 355
assign_input_words(228, 127);
assign_carry_in('0');

--N + 127 + 1 = 356
assign_input_words(228, 127);
assign_carry_in('1');

--M = 27
--N + M + 1 = 256
assign_input_words(228, 27);
assign_carry_in('1');

--M = 26
--N + M + 0 = 255
assign_input_words(228, 26);
assign_carry_in('1');

--outros testes
--N + 10 + 0 = 238
assign_input_words(228, 10);
assign_carry_in('0');

--N + 0 + 0 = 228
assign_input_words(228, 0);
assign_carry_in('0');
```

p.1) Faça o upload da imagem da carta de tempos da simulação feita, com todos os portos (com a devida resolução). Faça anotações na figura que deixem aparentes os resultados da sua simulação com o testbench (apoiados nas explicações do item p.2).

- Soma 1: 53 + 127 + 0 = 180
  - Resultado Simulação = 180

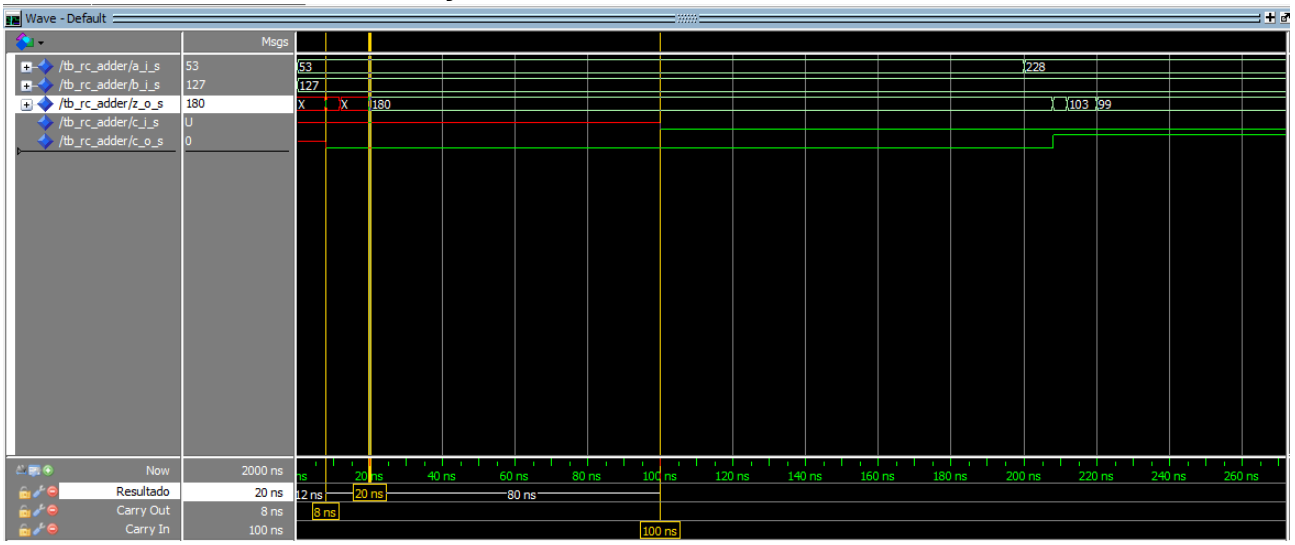

Figura 1: Soma 1: 53 + 127 + 0 = 180

- Soma 2: N + 127 + 0 = 355
  - Resultado Simulação = 99

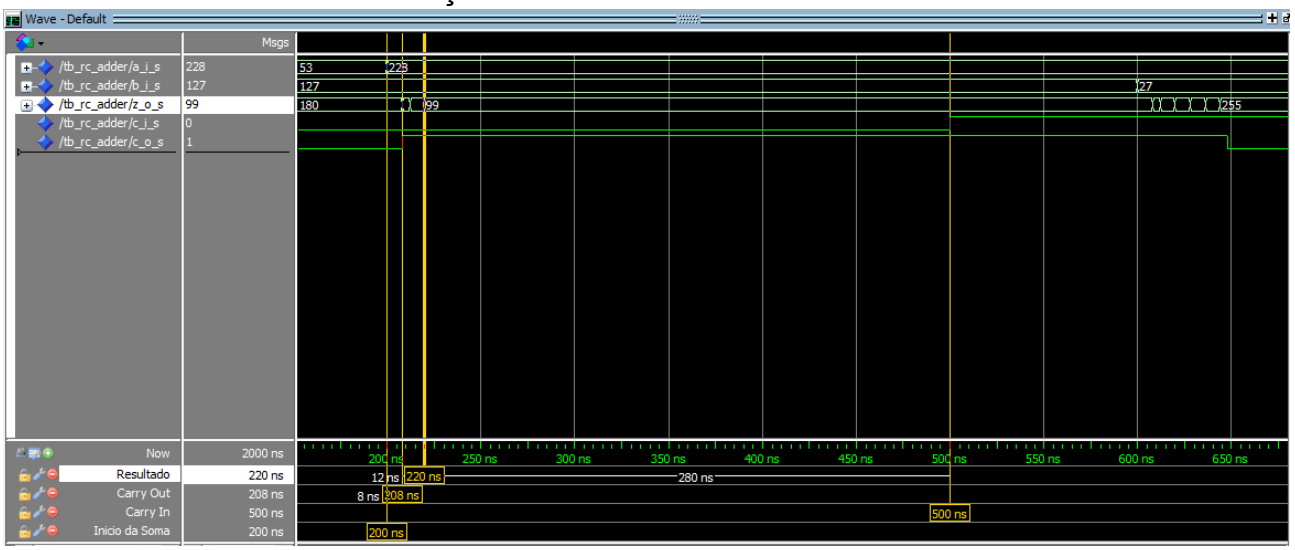

Figura 2: Soma 2: N + 127 + 0 = 355

- Soma 3: N + 127 + 1 = 356
  - Resultado Simulação = 99

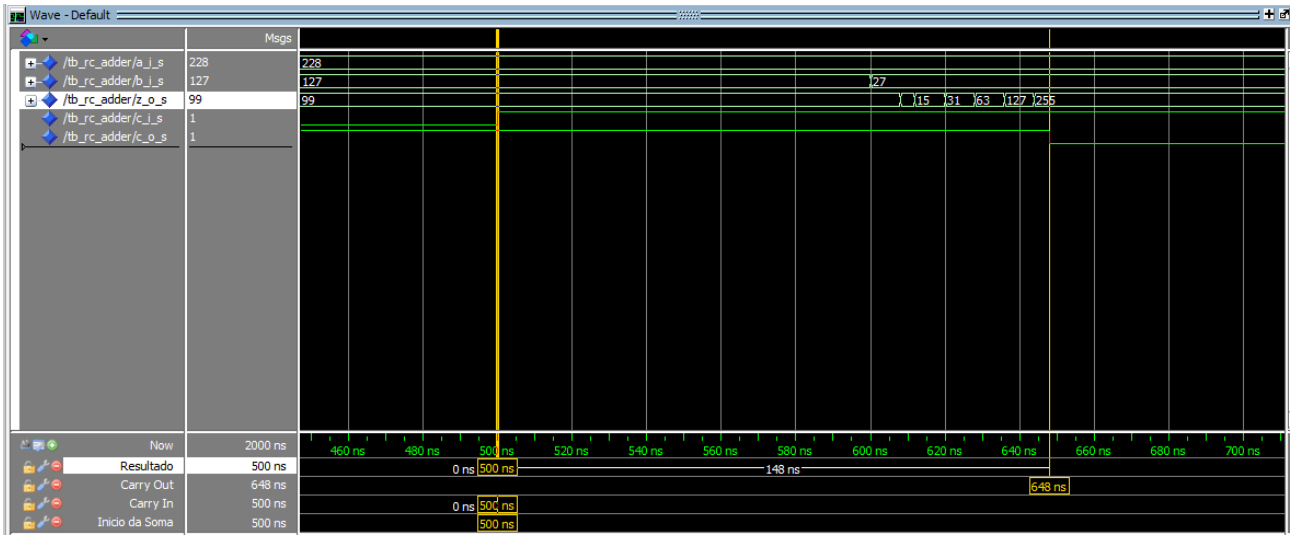

Figura 3: Soma 3: N + 127 + 1 = 356

- Soma 4: N + M + 1 = 256
  - Resultado Simulação = 127

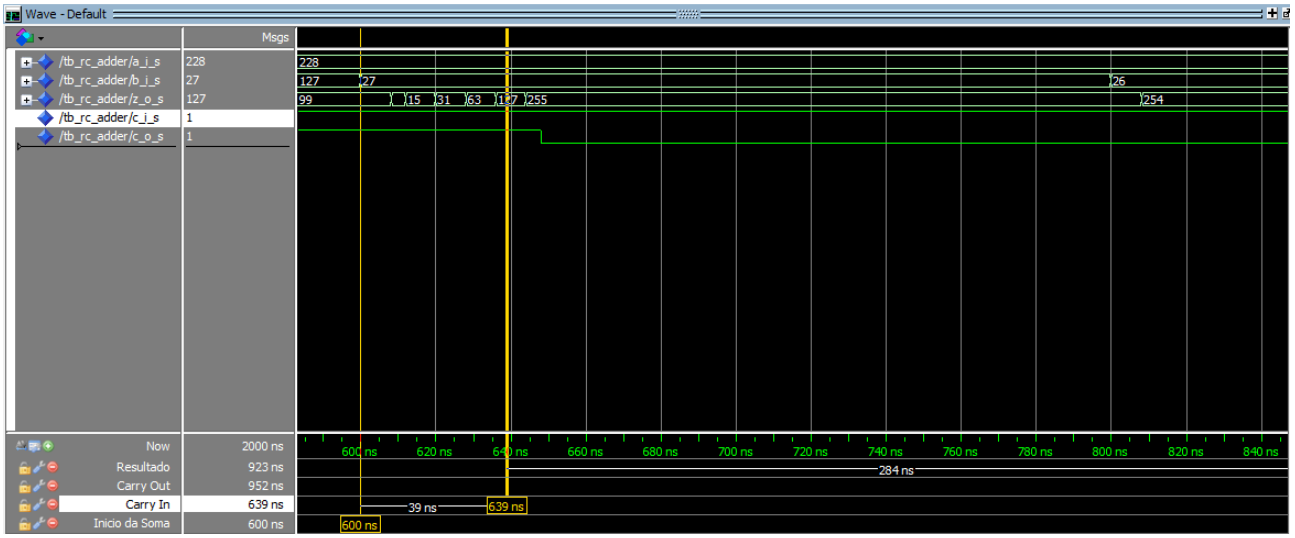

Figura 4: Soma 4: N + M + 1 = 256

- Soma 5: N + M + 0 = 255
  - Resultado Simulação = 255

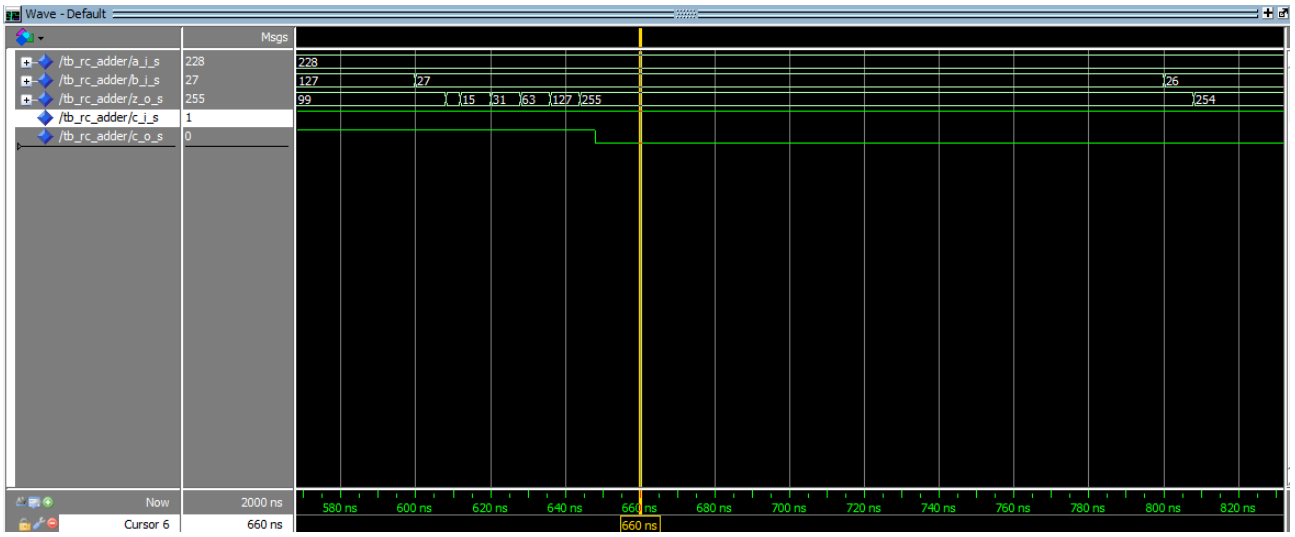

Figura 5: Soma 5: N + M + 0 = 255

- Soma 6: N + 10 + 0 = 238
  - Resultado Simulação = 238

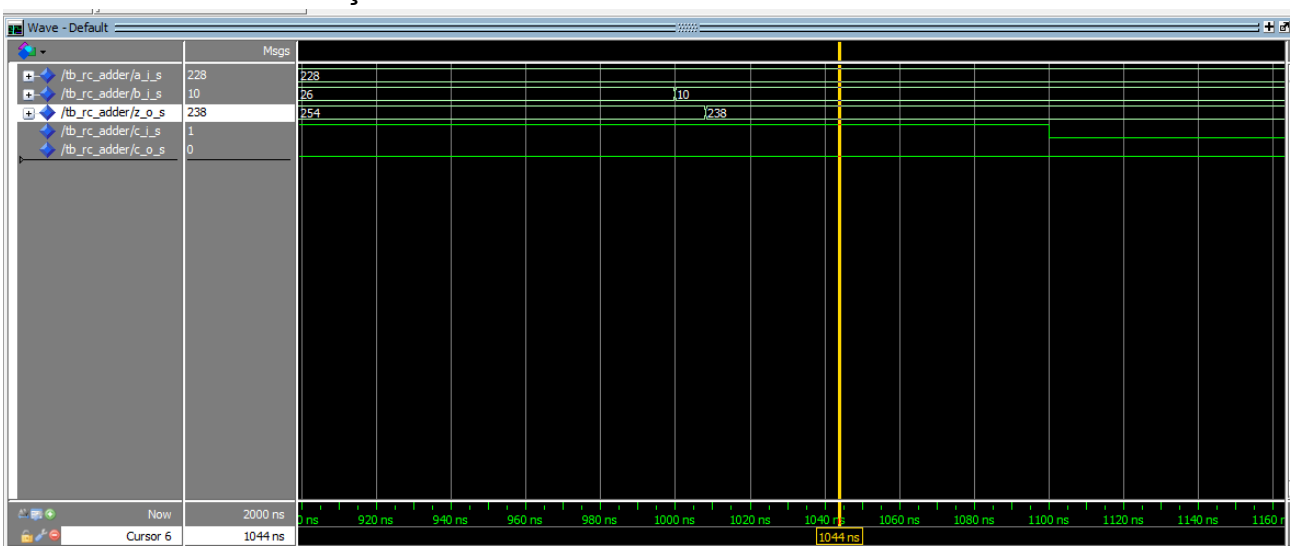

Figura 6: Soma 6: N + 10 + 0 = 238

- Soma 7: N + 0 + 0 = 228
  - Resultado Simulação = 228

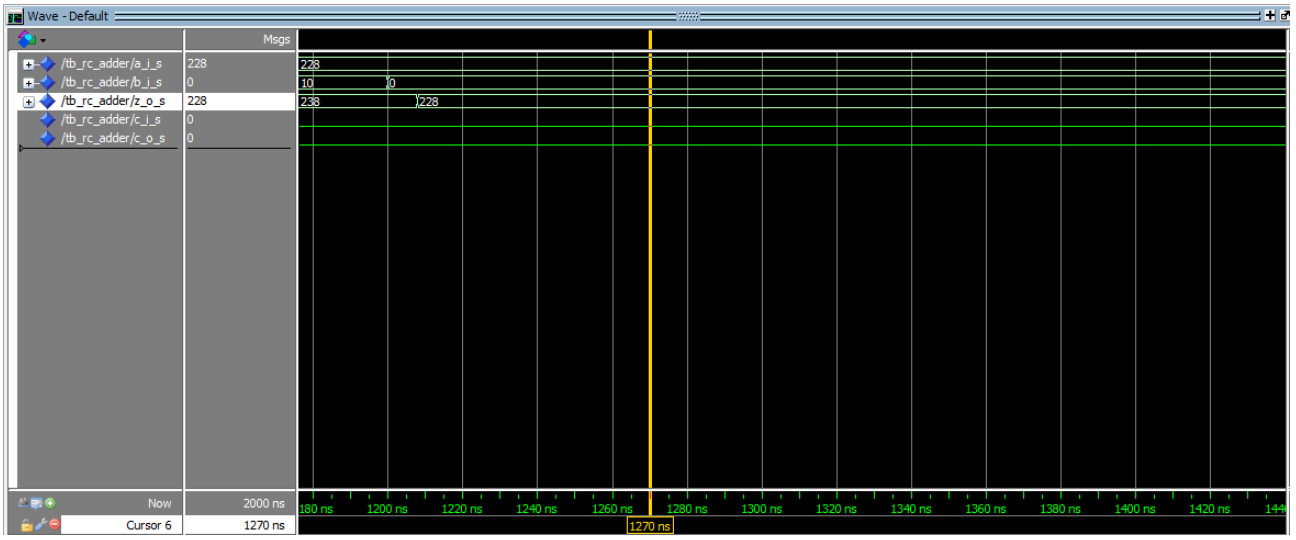

Figura 7: Soma 7: N + 0 + 0 = 228

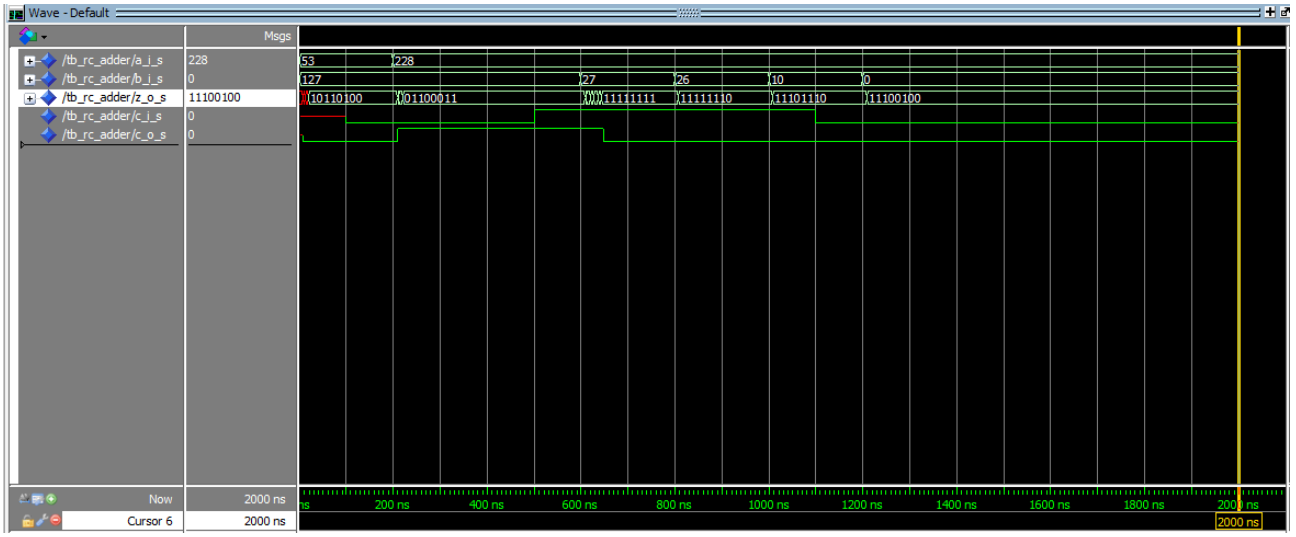

Figura 8: Simulação Completa

p.2) No campo abaixo, baseado nas anotações na figura do item p.1), discuta por que os resultados das somas são os esperados .

Para este somador, o resultado máximo é (2^8-1) = 255. Assim sendo, todos os resultados que ultrapasse esse limite possuem resultados “99”. Isso se dá porque a soma 2 e a soma 3 amostra no vetor de resultado os 8 bits menos significativos, sendo necessários o uso do bit de carry final (carry out) para amostrar o resultado correto:

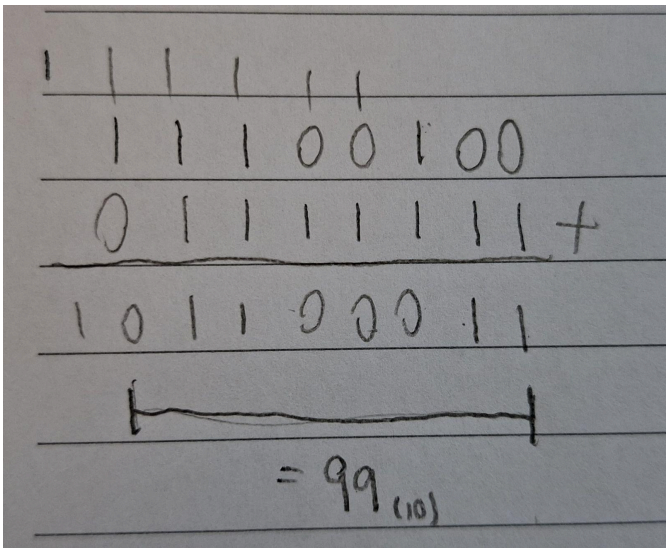

Figura 9: Soma 2 em binário

Para as somas menores que 256 (8 bits), os resultados foram amostrados corretamente. Dessa forma, os resultados obtidos foram idênticos ao esperado.

2) ALU

h.1) Insira no quadro abaixo o trecho do <u>módulo stimuli_module</u> proposto, com o **procedure** para a aplicação dos estímulos à ULA .

```
--teste 1: seleção do mux
            one_op <= N;
            rb_op <= std_logic_vector(to_unsigned(0,WIDTH));
            alu_ctrl <= '1'; --soma selecionada
            wait for TIME_DELTA;

            --teste 2: seleção do mux
            alu_ctrl <= '0'; --rb_op selecionada
            wait for TIME_DELTA;
```

h.2) Insira no quadro abaixo o trecho do <u>módulo stimuli_module</u> proposto em que o **procedure** acima **é utilizado**.

```
--teste 3: soma com incrementos unitários em rb_op
            teste3 : for incremento in 1 to 27 loop
                one_op <= std_logic_vector(to_unsigned(228+incremento,WIDTH));
                alu_ctrl <= '1';
                wait for TIME_DELTA;

                alu_ctrl <= '0';
                wait for TIME_DELTA;
            end loop teste3;
```

j.1) Faça o upload da imagem da carta de tempos da simulação feita, com todos os os portos (com a devida resolução). <u>Faça anotacões na</u> figura que deixem aparentes os resultados da sua simulação com o testbench (apoiados nas explicações do item j.2).

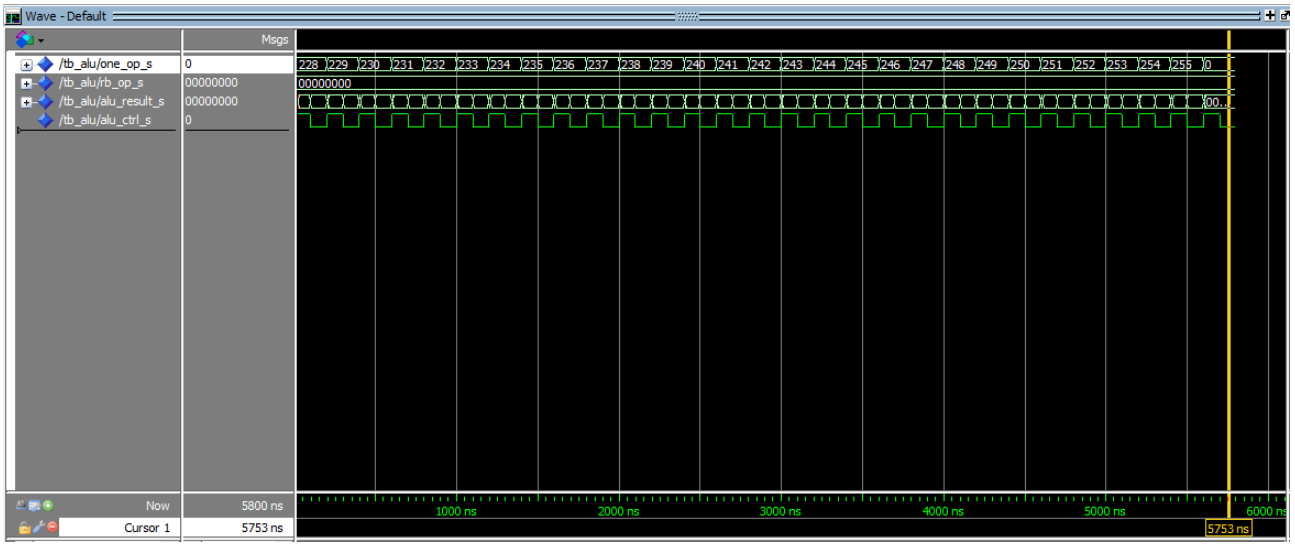

Figura 10: Simulação Completa

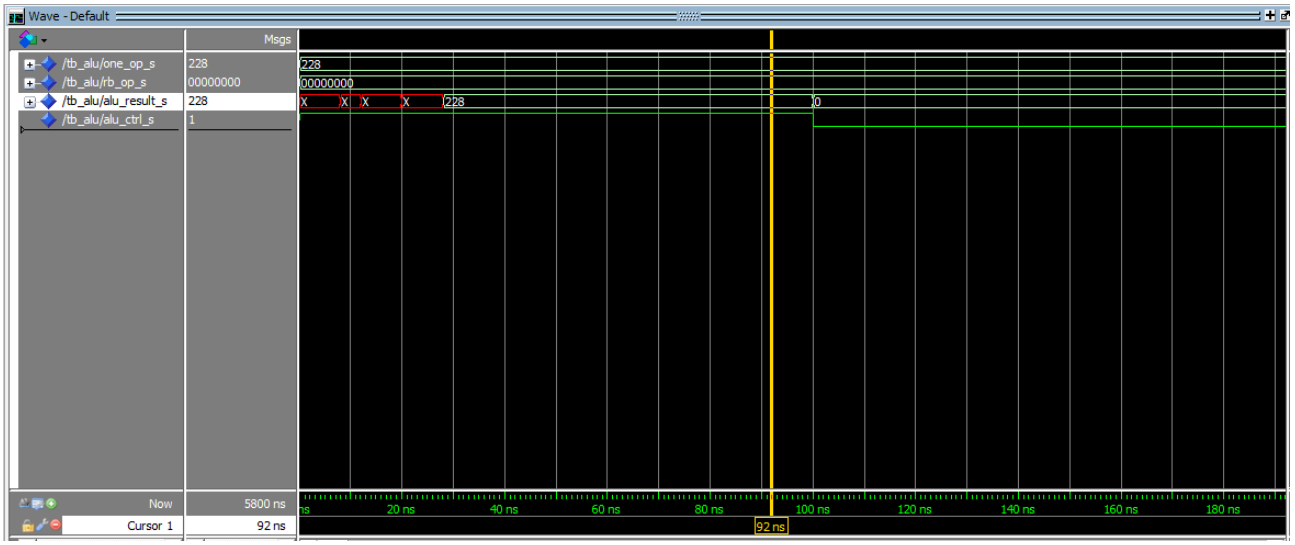

Figura 11: Teste 1

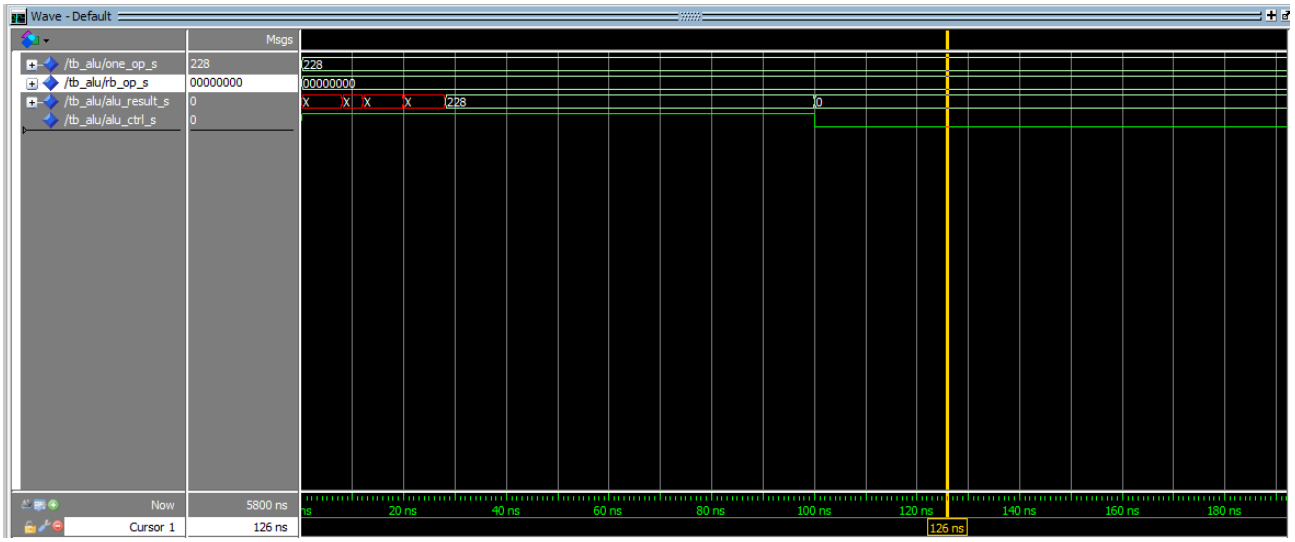

Figura 12: Teste 2

Figura 13: Teste 3

j.2) No campo abaixo, baseado nas anotações na figura do item j.1), discuta por que os resultados da ALU estariam corretos .

Para os testes 1 e 2, o esperado era o código agir como um multiplexador, de modo a selecionar a entrada rb_op caso o pino de seleção alu_ctrl fosse 0/LOW ou a entrada que representa o resultado da soma entre o rb_op com o one_op. Já no teste 3, foram realizadas incrementações do one_op sucessivamente até exceder o tamanho máximo (255 - 8 bits) do código e a saída ser o próximo valor após 255, que no caso é 256 = 100000000, em que é mostrado somente os 8 bits menos significativos por conta do tamanho da saída.

3)

m) Insira no quadro abaixo os trechos da arquitetura do módulo stimuli_module, em que fiquem identificadas e comentadas todas as mudanças por você realizadas para a verificação do contador (mostre todos os itens).

```
begin
        -- Apply test vectors

        reset_activate;   -- counter will start

        assign_flags ('1', '1');                -- as contagens est►o
habilitadas

        wait for 1*CLK_PERIOD;


            assign_input_words (SPEED'VAL(0));     -- ► equvivalente a
"assign_input_words (x4)" - Por que?

            while (cnt_guru_rdy /= '1' )
                loop

                    wait for 1*CLK_PERIOD;
            end loop;



        wait for 50*CLK_PERIOD;



        --assign_flags ('1', '1');              -- as contagens est►o
habilitadas

        --wait for 1*CLK_PERIOD;


            assign_input_words (SPEED'VAL(1));     -- ► equvivalente a
"assign_input_words (x1)" - Por que?

            while (to_integer(cnt_value) /= 228) loop
                wait for 1*CLK_PERIOD;
            end loop;
            reset_activate;
```

```
            wait for 50*CLK_PERIOD;


        --assign_flags ('0', '1');              -- as contagens est►o
habilitadas

        --wait for 1*CLK_PERIOD;

        assign_input_words (SPEED'VAL(2));
        while (to_integer(cnt_value) /= 456) loop
            wait for 1*CLK_PERIOD;
        end loop;
        reset_activate; -- Dispara o reset na hora exata em que atinge 456
        wait for 50*CLK_PERIOD;

        assign_input_words (SPEED'VAL(3));
        while (cnt_guru_rdy /= '1') loop
            wait for 1*CLK_PERIOD;
        end loop;
        wait for 50*CLK_PERIOD;


        wait;


    end process sim;
```

n.1) Faça o upload da(s) imagem(ns) da carta de tempos da simulação feita, com os sinais adequados. Faça as anotações necessárias para evidenciar as ações dos resets; apoiando-se nas explicações do item n.2).

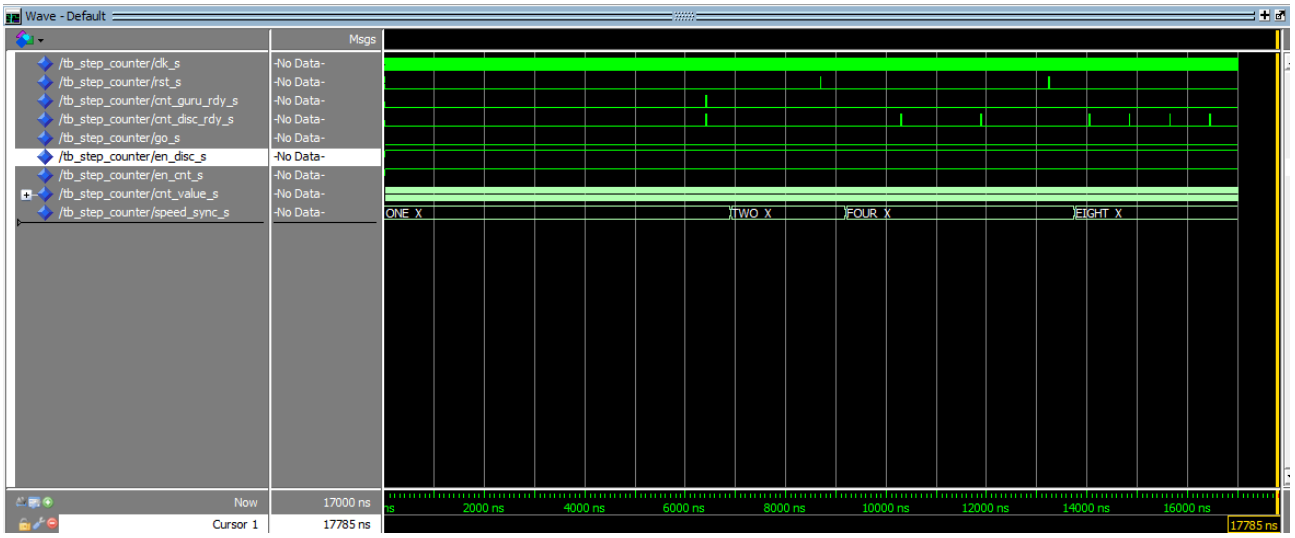

Figura 14: Simulação Completa

n.2) No campo abaixo, interprete a(s) imagem(ns) do item n.1), mostrando como fica evidente o momento correto dos resets..

Inicialmente, a simulação está na velocidade base (ONE_X) e realiza a contagem até atingir o valor limite de 640, realizar um reset e, após 50 ciclos de clock, passar para o estágio de velocidade TWO_X. Nesse estágio, ao atingir o valor de N=248, ocorre outro reset e após outros 50 ciclos de clock, a velocidade aumenta para FOUR_X. Nesse, ao atingir a contagem de 2N, ocorre outro reset e após 50 ciclos de clock, entra no estágio final de EIGHT_X.